

PFLICHTENHEFT

Zugverwaltungssystem

Fahrkartenverkauf & Zügeinsatzverwaltung

Projekttitle	Zugverwaltungssystem
Datum	Mai 2026

Zielbestimmung

Das Zugverwaltungssystem dient der digitalen Verwaltung von Zügen, Waggons, Linien, Bahnhöfen, Fahrgästen und Fahrkarten. Es stellt eine webbasierte REST-API sowie ein browserbasiertes Frontend bereit.

Mussziele

- Anlegen, Anzeigen und Löschen von Zügen und Waggons
- Zuordnung von Waggons zu Zügen
- Erfassung von Fahrgästen im System
- Kauf von Fahrkarten mit Zuordnung zu Zug, Waggon, Fahrgast und Abfahrtszeit
- Verwaltung von Linien und zugehörigen Bahnhöfen
- Zuordnung von Zügen zu Linien
- Automatische Reduktion verfügbarer Plätze bei Fahrkartenkauf
- Anzeige aller gebuchten Fahrkarten
- Stornierung von Fahrkarten
- Persistierung aller Daten in einer relationalen Datenbank
- REST-API mit JSON-Datenformat

Wunschziele

- Browserbasiertes Frontend zur Bedienung ohne technische Kenntnisse
- Filterung und Suche von Zügen, Fahrkarten und Fahrgästen
- Auslastungsanzeige je Waggon
- Trennung von Admin und Standard-User

2. Produkteinsatz

2.1 Anwendungsbereich

Das System richtet sich an Eisenbahn- oder Nahverkehrsbetriebe, die eine einfache, zentrale Lösung zur Verwaltung ihrer Zugflotte und des Fahrkartenwesens benötigen. Es eignet sich für den internen Betrieb durch Disponenten und Betriebspersonal.

2.2 Zielgruppen

Benutzergruppe	Aufgaben	Technische Anforderungen
Disponent / Administrator	Zuege & Waggons verwalten, Linien anlegen	Webbrowser, Netzwerkzugang zum Server
Fahrkartenpersonal	Fahrgaeste anlegen, Fahrkarten kaufen	Webbrowser, Netzwerkzugang zum Server
Entwickler / Systemintegrator	API-Integration, Datenbankpflege	REST-Client, Java/MySQL-Kenntnisse

2.3 Betriebsbedingungen

- Betrieb als lokaler oder netzwerkgebundener Java-Prozess (Spring Boot Embedded Tomcat)
- MySQL-Datenbank erforderlich (Verbindungsdaten konfigurierbar via application.properties)
- Frontend laeuft im Webbrowser ohne Installation (statisches HTML/JavaScript)
- Betrieb auf handelsüblicher Server-Hardware ausreichend

3. Produktübersicht

3.1 Systemarchitektur

Das System folgt einer klassischen dreischichtigen Architektur:

- Präsentationsschicht: Statisches HTML/CSS/JavaScript-Frontend (index.html), das über REST-Aufrufe mit dem Backend kommuniziert
- Anwendungsschicht: Spring Boot-Applikation mit Controller-, Service- und Modellklassen
- Datenschicht: ORMLite-ORM über JDBC-Verbindung zu einer MySQL-Datenbank

3.2 Systemkomponenten

Komponente	Technologie	Beschreibung
ZugverwaltungApplication	Spring Boot	Einstiegspunkt, startet den eingebetteten Webserver
DatabaseConfig	ORMLite / JDBC	Erstellt alle DAOs und Tabellen beim Start
ZugController	Spring REST	REST-Endpunkte fuer Zuege und Waggons
TicketController	Spring REST	REST-Endpunkte fuer Fahrgaeste und Fahrkarten
LinieController	Spring REST	REST-Endpunkte fuer Linien (ausbaufaehig)
ZugService	Java	Geschaeftslogik fuer Zug- und Waggonverwaltung inkl. kaskadierten Loeschungen
FahrkartenService	Java	Geschaeftslogik Fahrkartenkauf inkl. Platzvalidierung
LinieService	Java	Verwaltung von Linien und Bahnhofen
Modelle (Zug, Waggon, Fahrkarte, Fahrgast, Linie, Bahnhof)	ORMLite Entities	Datenbankentitaeten mit Foreign-Key-Beziehungen

4. Funktionale Anforderungen

4.1 Zugverwaltung

/FA-Z-01/ Zug anlegen

Das System muss es ermöglichen, einen neuen Zug mit einem Namen anzulegen. Der Zug wird in der Datenbank persistiert und erhält eine automatisch generierte ID.

Endpunkt: *POST /api/zuege*

/FA-Z-02/ Alle Züge anzeigen

Das System muss eine Liste aller gespeicherten Züge inklusive ihrer Waggonanzahl zurückgeben.

Endpunkt: *GET /api/zuege*

/FA-Z-03/ Zug löschen

Das System muss einen Zug anhand seiner ID löschen. Beim Löschen werden alle zugehörigen Waggonen sowie alle Fahrkarten dieser Waggonen automatisch mitgelöscht (kaskadiertes Löschen).

Endpunkt: *DELETE /api/zuege/{zugId}*

4.2 Waggonverwaltung

/FA-W-01/ Waggon anlegen und Zug zuweisen

Das System muss das Anlegen eines Waggonen mit Bezeichnung und Sitzplatzzahl sowie dessen Zuweisung zu einem bestehenden Zug ermöglichen.

Endpunkt: *POST /api/zuege/{zugId}/waggonen*

/FA-W-02/ Waggonen eines Zuges anzeigen

Das System muss alle Waggonen eines bestimmten Zuges abrufbar machen, inklusive der aktuell verfügbaren Plätze.

Endpunkt: *GET /api/tickets/waggonen?zugId={id}*

/FA-W-03/ Waggon löschen

Das System muss das Löschen eines Waggonen anhand seiner ID ermöglichen.

Endpunkt: *DELETE /api/zuege/{waggonId}/waggonen*

4.3 Fahrgastverwaltung

/FA-F-01/ Fahrgast anlegen

Das System muss das Anlegen eines neuen Fahrgasts mit Name ermöglichen.

Endpunkt: *POST /api/tickets/fahrgaeste*

/FA-F-02/ Alle Fahrgäste anzeigen

Das System muss eine Liste aller gespeicherten Fahrgäste zurückgeben.

Endpunkt: *GET /api/tickets/fahrgaeste*

4.4 Fahrkartenverkauf

/FA-T-01/ Fahrkarte kaufen

Das System muss den Kauf einer oder mehrerer Fahrkarten ermöglichen. Dabei werden Zug, Waggon, Fahrgast, Anzahl der Plätze und Abfahrtszeit angegeben. Das System prüft:

- Ob der gewählte Waggon zum angegebenen Zug gehört
- Ob ausreichend Plätze im Waggon verfügbar sind

Bei Erfolg wird die verfügbare Platzzahl im Waggon entsprechend reduziert.

Endpunkt: *POST /api/tickets/buy* (Query-Parameter: *zugId, waggonId, fahrgastId, anzahl, abfahrtszeit*)

/FA-T-02/ Alle Fahrkarten anzeigen

Das System muss eine Liste aller gebuchten Fahrkarten mit vollständigen Informationen zu Zug, Waggon, Fahrgast und Abfahrtszeit zurückgeben.

Endpunkt: *GET /api/tickets*

4.5 Linienverwaltung

/FA-L-01/ Linie anlegen

Das System muss das Anlegen einer Linie mit Bezeichnung ermöglichen. (Implementiert in LinieService, REST-Endpunkte im LinieController noch nicht ausgebaut.)

/FA-L-02/ Bahnhof anlegen

Das System muss das Anlegen von Bahnhöfen und deren Zuordnung zu einer Linie ermöglichen.

5. Nichtfunktionale Anforderungen

5.1 Benutzbarkeit

- Das Frontend muss ohne Installation direkt im Webbrowser bedienbar sein
- Alle Aktionen geben dem Benutzer unmittelbares Feedback (Toast-Benachrichtigungen)
- Waggons zeigen einen visuellen Auslastungsbalken; kritisch niedrige Verfügbarkeit (< 10 Plätze) wird farblich hervorgehoben
- Bedienung über intuitive Formulare und Listen ohne technische Vorkenntnisse

5.2 Datenkonsistenz

- Alle Schreiboperationen werden atomar in der Datenbank durchgeführt
- Beim Löschen eines Zuges werden alle abhängigen Datensätze (Waggons, Fahrkarten) kaskadiert entfernt
- Die Platzbelegung eines Waggons wird nach jedem Fahrkartenkauf sofort aktualisiert
- Ungültige Operationen (z. B. zu wenig Plätze, falscher Waggon) werden mit aussagekräftigen Fehlermeldungen abgelehnt

5.3 Wartbarkeit und Erweiterbarkeit

- Klare Trennung von Controller-, Service- und Modellschicht (MVC-Muster)
- Dependency Injection über Spring Boot für einfachen Austausch von Komponenten
- ORMLite als leichtgewichtiges ORM ermöglicht einfache Datenbankmigrationen
- Konfigurierbare Datenbankverbindung über Umgebungsvariablen (db.url, db.username, db.password)

5.4 Technische Anforderungen

Merkmal	Ausprägung
Backend-Framework	Spring Boot (Java)
ORM	ORMLite (com.j256.ormlite)
Datenbank	MySQL (konfigurierbar via JDBC-URL)
API-Format	REST / JSON (Jackson)
Frontend	HTML5 / CSS3 / Vanilla JavaScript
Datums-/Zeitformat	ISO 8601 / LocalDateTime (Java)
Deployment	Embedded Tomcat (Spring Boot JAR)

6. Datenmodell

6.1 Entitäten und Attribute

Entitaet	DB-Tabelle	Schluessel-Attribute	Beschreibung
Zug	zuege	id, name	Ein Zug besteht aus mehreren Waggons
Waggon	waggons	id, bezeichnung, plaetze, zug_id	Gehört zu genau einem Zug, haelt verfügbare Plätze
Fahrgast	fahrgaeste	id, name	Passagier, dem Fahrkarten zugeordnet werden
Fahrkarte	fahrkarten	id, plaetze, zug_id, waggon_id, fahrgast_id, abfahrtszeit	Buchungsbeleg, verbindet Fahrgast, Waggon, Zug und Zeit
Linie	linien	id, bezeichnung	Strecke mit mehreren Bahnhöfen und Zügen
Bahnhof	bahnhoefe	id, bezeichnung, linie_id (als zug_id gespeichert)	Haltepunkt auf einer Linie

6.2 Beziehungen

- Zug (1) → (N) Waggon: Ein Zug hat mehrere Waggons (ForeignCollection in ORMLite)
- Waggon (1) → (N) Fahrkarte: Ein Waggon kann mehreren Fahrkarten zugeordnet sein
- Fahrgast (1) → (N) Fahrkarte: Ein Fahrgast kann mehrere Fahrkarten besitzen
- Zug (1) → (N) Fahrkarte: Eine Fahrkarte referenziert direkt den Zug
- Linie (1) → (N) Bahnhof: Eine Linie besteht aus mehreren Bahnhöfen

6.3 Bekannte Hinweise zum Datenmodell

- In der Klasse Bahnhof ist das Fremdschlüssel-Feld zur Linie als 'zug_id' benannt (Inkonsistenz, Refactoring empfohlen)
- Die Klasse Linie deklariert ForeignCollections auf Zug und Bahnhof, obwohl Zug keine direkte Fremdschlüssel-Spalte zur Linie besitzt
- Das Feld 'anzahlwaggons' in Zug wird in der Datenbank gespeichert, aber zur Laufzeit dynamisch aus der ForeignCollection ermittelt
- Die Tabellenspalte 'abfahrtszeit' in 'fahrkarten' wird per ALTER TABLE nachträglich hinzugefügt (Migrations-Workaround)

7. REST-API Übersicht

Methode	Endpunkt	Controller	Beschreibung
GET	/api/zuege	ZugController	Alle Zuege auflisten
POST	/api/zuege	ZugController	Neuen Zug anlegen
DELETE	/api/zuege/{zugId}	ZugController	Zug loeschen (kaskadiert)
GET	/api/zuege/waggons	ZugController	Alle Waggons auflisten
POST	/api/zuege/{zugId}/waggons	ZugController	Waggon zu Zug hinzufuegen
DELETE	/api/zuege/{waggonId}/waggons	ZugController	Waggon loeschen
GET	/api/tickets	TicketController	Alle Fahrkarten auflisten
POST	/api/tickets/buy	TicketController	Fahrkarte kaufen
GET	/api/tickets/waggons?zugId=	TicketController	Waggons eines Zuges abrufen
GET	/api/tickets/fahrgaeste	TicketController	Alle Fahrgaeste auflisten
POST	/api/tickets/fahrgaeste	TicketController	Fahrgast anlegen

8. Benutzeroberfläche (Frontend)

8.1 Struktur

Das Frontend ist eine einzelne HTML-Seite (index.html) mit vier Tabs:

Tab	Inhalt / Funktionen
Zuege	Zug anlegen, alle Zuege anzeigen (mit Waggonanzahl), Zug loeschen
Waggons	Waggon anlegen und Zug zuweisen, alle Waggons mit Auslastungsbalken anzeigen, Waggon loeschen
Fahrgaeste	Fahrgast anlegen, alle Fahrgaeste anzeigen
Tickets	Fahrkarte kaufen (Zug, Waggon, Fahrgast, Anzahl, Abfahrtszeit), alle Buchungen anzeigen

8.2 Design

- Dunkles Farbschema (#0e0f11 Hintergrund) mit Akzentfarbe Gelb-Gruen (#e8ff57)
- Schriftarten: Syne (Ueberschriften) und DM Mono (Daten/Code)
- Animierter Statusindikator (pulsierende gruene Anzeige)
- Toast-Benachrichtigungen fuer Erfolgsmeldungen und Fehlerfaelle
- Responsives Grid-Layout fuer Formular und Listenbereich
- Auslastungsbalken je Waggon mit Farbwechsel bei geringer Verfuegbarkeit

9. Fehler- und Ausnahmebehandlung

Fehlerszenario	Verhalten des Systems
Waggon gehoert nicht zum angegebenen Zug	SQL-Exception mit Fehlermeldung: Waggon gehoert nicht zu Zug
Nicht genugend Plaetze im Waggon	SQL-Exception mit Meldung ueber verfuegbare Platzzahl
Zug / Waggon / Fahrgast nicht gefunden	Fehlerstring an den Client (HTTP 200, Textantwort)
Datenbankfehler allgemein	Fehlermeldung wird als String an den Client zurueckgegeben
Loeschen eines Zuges mit Abhaengigkeiten	Kaskadiertes Loeschen von Waggonen und Fahrkarten vor dem Zug

Hinweis: Die aktuelle Fehlerbehandlung gibt Fehler als HTTP-200-Antwort mit Textinhalt zurück. Für eine produktionsreife API wird die Verwendung standardisierter HTTP-Fehlercodes (4xx/5xx) empfohlen.

10. Offene Punkte & Empfehlungen

10.1 Bekannte Mängel

- LinieController ist leer: Die Linienverwaltung hat keine REST-Endpunkte und ist über das Frontend nicht erreichbar
- LinieService fehlt die @Service-Annotation: Dadurch kann der Service nicht automatisch durch Spring injiziert werden
- Feldname 'zug_id' in Bahnhof-Entitaet ist irrefuehrend (sollte 'linie_id' heissen)
- Fehlerrueckgaben als HTTP 200 statt korrekter HTTP-Fehlercodes (400, 404, 409)
- Keine Eingabevalidierung (Bean Validation / @Valid) auf Controller-Ebene
- Keine Authentifizierung oder Autorisierung implementiert
- Datenbankmigrationen werden durch try/catch-Blocke mit ALTER TABLE behelfsmaessig realisiert

10.2 Empfohlene Erweiterungen

- Implementierung der REST-Endpunkte fuer Linien und Bahnhoefe im LinieController
- Einfuehrung von Spring Security fuer Benutzerverwaltung und Zugriffsschutz
- Migration von ORMLite-Workarounds zu Flyway oder Liquibase fuer Datenbankmigrationen
- Einfuehrung von DTO-Klassen (Data Transfer Objects) zur Entkopplung von Datenbank- und API-Schicht
- Unit- und Integrationstests mit JUnit und MockMvc
- Swagger/OpenAPI-Dokumentation der REST-API
- Containerisierung mit Docker fuer einfaches Deployment
- Stornierungsfunktion fuer Fahrkarten inkl. Rueckgabe der Plaetze an den Waggon

Ende des Pflichtenhefts

Zugverwaltungssystem – Version 1.0 – Mai 2026